

GEF Secure

Implementación del Escaneo de Vulnerabilidades por Activo, Entorno y Global

Documento técnico de implementación para backend y frontend: dispara y hace seguimiento de los 3 tipos de escaneo que el workflow de n8n ya soporta internamente, cerrando la brecha identificada en el apéndice del documento de automatización de n8n.

Arquitecto de Software Senior · Backend & Frontend
Proyecto GEF Secure — Tesis UTN FRM
Documento generado el 2026-08-02

Índice

- 1. Resumen ejecutivo**
- 2. Estado actual verificado**
- 3. Evidencia del contrato en el workflow n8n**
- 4. Arquitectura objetivo**
- 5. Implementación backend**
 - 5.1 N8nWebhookService — contrato unificado
 - 5.2 Manejo de errores
 - 5.3 Endpoint scan por activo (fix)
 - 5.4 Endpoint scan por entorno (nuevo)
 - 5.5 Endpoint scan global (nuevo)
 - 5.6 Endpoint de lectura para polling
- 6. Implementación frontend**
 - 6.1 Tipos y API client
 - 6.2 Hook de polling unificado
 - 6.3 Inventario: fix del scan por activo + scan por entorno
 - 6.4 Dashboard: scan global
- 7. Procedimiento de implementación**
- 8. Validación (checklist)**
- 9. Resolución de problemas**
- 10. Fuera de alcance / mejoras futuras**
- 11. Conclusión**

1. Resumen ejecutivo

El documento `GEF_Secure_Automatizacion_n8n.pdf` dejó, en su apéndice (sección 14), un contrato definido pero no implementado: el workflow de n8n ya sabe rutear un scan por **activo**, por **entorno** o **global** a partir de dos campos en el body del webhook (`activo_name`, `entorno`), pero el backend solo implementa el primer caso — y con un bug de payload que ni siquiera ese caso deja funcional hoy. Este documento cierra esa brecha: especifica, con código listo para aplicar, cómo el backend dispara y el frontend hace seguimiento de los 3 tipos de scan.

Todo lo que sigue está verificado contra el código real del proyecto (controllers, services, repositories, DTOs y componentes React ya existentes, leídos directamente) y contra las queries reales del workflow (`Check_Internal_Assets` y `Persist_Scan_Statistics`), no supuesto. En particular, se confirma acá — por primera vez con evidencia directa del SQL — que:

- El campo `activo_name` debe matchear `assets.name` (no `software`, que es lo que el backend envía hoy).
- El campo `entorno` debe matchear `environments.name`.
- n8n persiste cada scan en `scan_reports` con `target_type` ∈ {`ACTIVO`, `ENTORNO`, `GLOBAL`} y `target_name` igual al valor efectivo enviado (o el literal `"TODO"` para `GLOBAL`) — esta es la clave para que el frontend pueda hacer polling de **los 3 casos con el mismo mecanismo**, algo que el documento anterior dejaba sin resolver.

Componente	Estado de partida (verificado)	Estado objetivo (este documento)
Scan por activo	Endpoint existe (<code>POST /api/assets/{id}/scan</code>) pero envía un payload que el workflow no lee (<code>software</code> en vez de <code>activo_name</code> =nombre del activo)	Payload corregido; error de n8n propagado como 502 en vez de 200 falso
Scan por entorno	No existe endpoint	<code>POST /api/environments/{id}/scan</code>
Scan global	No existe endpoint	<code>POST /api/scan</code>
Seguimiento del resultado	El frontend dispara y recarga la lista de inmediato — no espera a que n8n termine	Endpoint de lectura <code>GET /api/scan-reports/latest</code> + hook de polling único para los 3 casos

2. Estado actual verificado

Leído directamente del código, sin asumir nada:

Backend

- `AssetController.triggerScan()` llama a `assetService.triggerScan(id)` (solo actualiza `lastScan`) y después, en el controller mismo, vuelve a buscar el activo (`assetRepository.findById` — el "double load" ya señalado en el análisis estratégico) para llamar a `webhookService.triggerScan(id, software, version, ecosystem)`.
- `N8nWebhookService.triggerScan()` arma `{assetId, software, version, ecosystem}` y hace *swallow* de cualquier excepción (solo loggea, nunca la relanza) — el controller siempre devuelve 200, incluso si n8n no respondió.
- `EnvironmentController` tiene CRUD completo pero ningún endpoint de scan.
- `ScanReportRepository` solo tiene `findAllByOrderByExecutedAtDesc(Pageable)` — no hay forma de buscar el último reporte de un `target_type/target_name` puntual.
- No existe ningún `ScanController` ni endpoint para scan global.

Frontend

- `Inventory.tsx` → `handleScan()` llama a `assetApi.triggerScan(id)`, muestra un toast de éxito **inmediatamente** y recarga la lista — sin esperar a que n8n procese nada. El "polling acotado a 30s" que describe la sección 7 del documento de automatización de n8n **no está implementado en el código actual**; era una recomendación de diseño, no algo ya construido.
- `environmentApi` ya existe completo (CRUD), se usa hoy solo para poblar el combo de entorno en el formulario de activos.
- No existe página ni control de UI para "escanear un entorno completo" ni "escanear todo".
- `DashboardStatsDTO` no tiene ningún campo de estado de scan — el polling necesita un endpoint propio, no puede apoyarse en `/api/dashboard/stats`.

Hallazgo colateral, fuera de alcance de este documento. `DashboardService.getStats()` filtra críticas con `vulnRepository.countByPriority("CRITICAL")` (inglés), pero `VulnPriority` en `frontend/src/types/index.ts` y los valores reales en la base son español (`CRITICA`). Confirmado empíricamente: con datos semilla que incluyen 2 vulnerabilidades `CRITICA`, el Dashboard mostró `criticalVulnerabilities: 0`. No se toca en este documento porque no bloquea el scan por alcance, pero queda anotado para no confundirlo con un bug nuevo introducido acá.

3. Evidencia del contrato en el workflow n8n

El apéndice del documento anterior ya había identificado el contrato de alto nivel (`activo_name` / `entorno`, ambos con fallback `"TODOs"`). Para este documento se profundizó, leyendo directamente las queries SQL de los nodos que efectivamente *usan* esos valores.

Nodo `Check_Internal_Assets` — cómo se filtra

```
SELECT a.id, a.name AS asset_name, a.software, a.ecosystem, a.version,
       e.name AS env_name, e.business_criticality AS env_criticality
FROM assets a
JOIN environments e ON a.environment_id = e.id
WHERE a.software = '{{ $json.vulnerabilities.package.name }}'
AND a.ecosystem = '{{ $json.vulnerabilities.package.ecosystem }}'
AND (
    '{{ filtro_entorno }}' = 'TODOs' OR e.name = '{{ filtro_entorno }}'
)
AND (
    '{{ filtro_activo }}' = 'TODOs' OR a.name = '{{ filtro_activo }}'
);
```

Esto confirma, sin ambigüedad, contra qué columna matchea cada campo del payload: `activo_name` → `assets.name` (no `software`), `entorno` → `environments.name`.

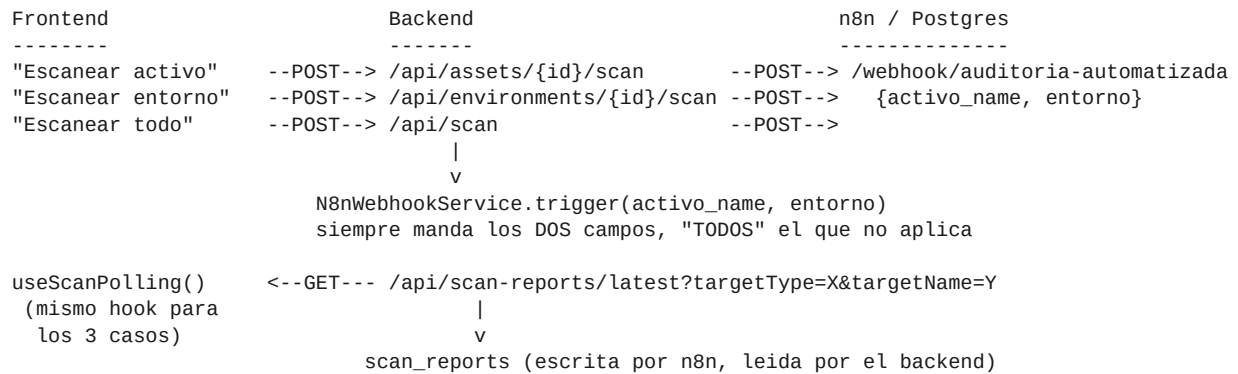
Nodo `Persist_Scan_Statistics` — cómo se guarda el resultado

```
INSERT INTO public.scan_reports (... , target_type, target_name)
VALUES (... ,
    '{{ filtro_activo != "TODOs" ? "ACTIVO"
      : (filtro_entorno != "TODOs" ? "ENTORNO" : "GLOBAL") }}',
    '{{ filtro_activo != "TODOs" ? filtro_activo
      : (filtro_entorno != "TODOs" ? filtro_entorno : "TODOs") }}'
);
```

Consecuencia directa para el diseño del polling (sección 5.6 y 6.2): los 3 casos escriben en la **misma tabla**, con la **misma convención** de dos columnas. Un único endpoint de lectura parametrizado por `target_type/target_name` alcanza para los 3 — no hace falta un mecanismo de polling distinto por caso, como sugería (sin resolver del todo) la sección 14.4 del documento anterior.

Caso	target_type	target_name
Scan por activo	ACTIVO	el <code>activo_name</code> enviado (= <code>assets.name</code>)
Scan por entorno	ENTORNO	el <code>entorno</code> enviado (= <code>environments.name</code>)
Scan global	GLOBAL	literal <code>"TODOs"</code>

4. Arquitectura objetivo



Se mantiene la decisión arquitectónica ya fijada en el documento de automatización: n8n escribe directo a Postgres, el backend nunca recibe datos de vuelta por HTTP de n8n. El único camino nuevo es de **lectura** (backend → Postgres → frontend), no cambia esa decisión.

5. Implementación backend

5.1 N8nWebhookService — contrato unificado

Reemplaza el método actual por tres métodos finos sobre una única implementación interna, que siempre manda los dos campos del contrato (sección 3):

```
// backend/src/main/java/com/gef/gefsecureapp/service/N8nWebhookService.java
package com.gef.gefsecureapp.service;

import com.gef.gefsecureapp.exception.N8nUnavailableException;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

import java.util.Map;

@Service
@RequiredArgsConstructor
@Slf4j
public class N8nWebhookService {

    private static final String TODOS = "TODOS";

    @Value("${n8n.webhook.url}")
    private String webhookUrl;

    private final RestClient.Builder restClientBuilder;

    public void triggerScanForAsset(String assetName) {
        trigger(assetName, TODOS);
    }

    public void triggerScanForEnvironment(String environmentName) {
        trigger(TODOS, environmentName);
    }

    public void triggerGlobalScan() {
        trigger(TODOS, TODOS);
    }

    private void trigger(String activoName, String entorno) {
        Map<String, Object> payload = Map.of(
            "activo_name", activoName,
            "entorno", entorno
        );
        try {
            restClientBuilder.build()
                .post()
                .uri(webhookUrl)
                .header("Content-Type", "application/json")
                .body(payload)
                .retrieve()
                .body(String.class);
            log.info("n8n webhook ok – activo_name={} entorno={}", activoName, entorno);
        } catch (Exception ex) {
            log.error("Error llamando webhook n8n (activo_name={}, entorno={}): {}",
                activoName, entorno, ex.getMessage());
            throw new N8nUnavailableException(
                "No se pudo disparar el scan en n8n: " + ex.getMessage(), ex);
        }
    }
}
```

```
}
```

Por qué siempre los dos campos, incluso hoy sin el fix de **BUG-N8N-07**. Como ya justificó la sección 14.3 del documento anterior: mandar **"TODOS"** explícito para el campo que no aplica evita depender de que n8n corrija el fallback que solo cubre el caso "no disparado por webhook". Con este contrato, los 3 endpoints funcionan aunque ese bug de n8n siga sin arreglar.

5.2 Manejo de errores

N8nUnavailableException (nueva, mismo patrón que **ConflictException** ya existente):

```
// backend/src/main/java/com/gef/gefsecureapp/exception/N8nUnavailableException.java
package com.gef.gefsecureapp.exception;

public class N8nUnavailableException extends RuntimeException {
    public N8nUnavailableException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Agregar en **GlobalExceptionHandler** (mismo patrón que **handleConflict**):

```
@ExceptionHandler(N8nUnavailableException.class)
public ResponseEntity<ErrorBody> handleN8nUnavailable(N8nUnavailableException ex) {
    log.error("n8n no respondió: {}", ex.getMessage());
    return ResponseEntity.status(HttpStatus.BAD_GATEWAY)
        .body(new ErrorBody(502, ex.getMessage()));
}
```

Nota honesta sobre el orden de operaciones. En el caso de scan por activo (§5.3), **assetService.triggerScan(id)** (que actualiza **lastScan**) corre en su propia transacción y ya queda confirmada en base cuando el controller llama al webhook. Si el webhook falla después, el cliente recibe 502 mostrando el error real — pero **lastScan** ya se actualizó. Es el mismo trade-off que ya aceptaba implícitamente el código actual; no lo introduce este documento. Corregirlo de raíz (no tocar **lastScan** hasta confirmar que n8n aceptó el scan) es un cambio de comportamiento más grande, fuera de alcance (ver sección 10).

Hallazgo verificado contra Docker real: sin timeout, **RestClient puede colgarse indefinidamente.**

Probando los 3 endpoints contra el stack completo, un **curl** disparando un scan quedó sin responder más de 120s. Diagnóstico: cero ejecuciones nuevas en n8n (el webhook ni llegó a registrar el intento) mientras que un **wget** directo desde el propio contenedor del backend contra el webhook de n8n respondía al instante — descartando un problema de red o de n8n. La causa real es que **RestClient.Builder** por default no tiene ningún límite de tiempo. Fix aplicado en **AppConfig**:

```
@Bean
public RestClient.Builder restClientBuilder() {
    SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(3_000);
    factory.setReadTimeout(5_000);
    return RestClient.builder().requestFactory(factory);
}
```

Con este bean ya inyectado en **N8nWebhookService** (§5.1), cualquier llamada que no obtenga respuesta de n8n en 3s de conexión o 5s de lectura termina en **N8nUnavailableException** → 502, en vez de bloquear el

hilo del request indefinidamente. Los valores son holgados, no ajustados al límite: en operación normal el webhook de n8n responde "onReceived" casi al instante.

5.3 Endpoint scan por activo (fix)

`AssetController.java` — reemplaza el método `triggerScan`:

```
@PostMapping("/{id}/scan")
@Operation(summary = "Disparar escaneo de un activo vía n8n")
public ResponseEntity<AssetDTO.Response> triggerScan(@PathVariable Long id) {
    AssetDTO.Response updated = assetService.triggerScan(id);
    webhookService.triggerScanForAsset(updated.getName());
    return ResponseEntity.ok(updated);
}
```

De paso elimina el "double load" ya señalado en el análisis estratégico: ya no hace falta `assetRepository.findById(id)` en el controller (se puede sacar el `AssetRepository` del constructor si no se usa en ningún otro método), porque `AssetDTO.Response` ya trae `name`.

5.4 Endpoint scan por entorno (nuevo)

`EnvironmentController.java` — agregar, inyectando `N8nWebhookService` en el constructor (ya usa `@RequiredArgsConstructor`, alcanza con declarar el campo `final`):

```
private final N8nWebhookService webhookService;

@PostMapping("/{id}/scan")
@Operation(summary = "Disparar escaneo de todos los activos de un entorno vía n8n")
public ResponseEntity<Void> triggerScan(@PathVariable Long id) {
    Environment env = environmentRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Environment", id));
    webhookService.triggerScanForEnvironment(env.getName());
    return ResponseEntity.accepted().build();
}
```

Devuelve `202 Accepted` (no `200 OK` con un recurso, porque no hay un único recurso actualizado — son N activos): comunica correctamente al cliente que la operación se aceptó pero es asíncrona, distinto semánticamente del 200 que sí tiene sentido en el scan por activo (donde el "recurso" es el propio activo con su `lastScan` actualizado).

5.5 Endpoint scan global (nuevo)

Nuevo archivo `ScanController.java` — se crea aparte de `EnvironmentController/AssetController` porque el scan global no pertenece conceptualmente a ninguno de los dos recursos:

```
// backend/src/main/java/com/gef/gefsecureapp/controller/ScanController.java
package com.gef.gefsecureapp.controller;

import com.gef.gefsecureapp.dto.ScanReportDTO;
import com.gef.gefsecureapp.repository.ScanReportRepository;
import com.gef.gefsecureapp.service.N8nWebhookService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
```

```

@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
@Tag(name = "Scan", description = "Disparo y seguimiento de escaneos globales")
public class ScanController {

    private final N8nWebhookService webhookService;
    private final ScanReportRepository scanReportRepository;

    @PostMapping("/scan")
    @Operation(summary = "Disparar escaneo global (todos los entornos, todos los activos) vía n8n")
    public ResponseEntity<Void> triggerGlobalScan() {
        webhookService.triggerGlobalScan();
        return ResponseEntity.accepted().build();
    }

    @GetMapping("/scan-reports/latest")
    @Operation(summary = "Último reporte de scan para un alcance (activo/entorno/global)")
    public ResponseEntity<ScanReportDTO> latestReport(
        @RequestParam String targetType,
        @RequestParam String targetName) {
        return scanReportRepository
            .findFirstByTargetTypeAndTargetNameOrderByExecutedAtDesc(targetType, targetName)
            .map(ScanReportDTO::from)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.noContent().build());
    }
}

```

5.6 Endpoint de lectura para polling

Agregar a `ScanReportRepository.java`:

```

java.util.Optional<ScanReport> findFirstByTargetTypeAndTargetNameOrderByExecutedAtDesc(
    String targetType, String targetName);

```

Nuevo `ScanReportDTO.java` — se agrega el método estático `from(ScanReport)` para no meter lógica de mapeo en el controller:

```

// backend/src/main/java/com/gef/gefsecureapp/dto/ScanReportDTO.java
package com.gef.gefsecureapp.dto;

import com.gef.gefsecureapp.model.ScanReport;
import lombok.*;
import java.time.LocalDateTime;

@Getter @Builder
public class ScanReportDTO {
    private LocalDateTime executedAt;
    private Integer totalDetected;
    private Integer criticals;
    private Integer highs;
    private Integer mediums;
    private Integer lows;
    private String systemStatus;
    private String targetType;
    private String targetName;

    public static ScanReportDTO from(ScanReport r) {
        return ScanReportDTO.builder()
            .executedAt(r.getExecutedAt())
            .totalDetected(r.getTotalDetected())
            .criticals(r.getCriticals())
            .highs(r.getHighs())

```

```
        .mediums(r.getMediums())
        .lows(r.getLows())
        .systemStatus(r.getSystemStatus())
        .targetType(r.getTargetType())
        .targetName(r.getTargetName())
        .build();
    }
}
```

Por qué 204 No Content y no 404 cuando todavía no hay ningún reporte para ese alcance. No es un error del cliente (el `targetType/targetName` son válidos) — es un estado legítimo: "todavía no corrió ningún scan de este alcance". El frontend lo interpreta como "seguir esperando", no como falla (ver §6.2).

6. Implementación frontend

6.1 Tipos y API client

`frontend/src/types/index.ts` — agregar:

```
export type ScanTargetType = 'ACTIVO' | 'ENTORNO' | 'GLOBAL'

export interface ScanReport {
  executedAt: string
  totalDetected: number
  criticals: number
  highs: number
  mediums: number
  lows: number
  systemStatus: string
  targetType: ScanTargetType
  targetName: string
}
```

`frontend/src/services/api.ts` — agregar (y sumar `ScanReport` al import de tipos):

```
export const scanApi = {
  triggerEnvironment: (id: number) => http.post<void>(`/environments/${id}/scan`),
  triggerGlobal: () => http.post<void>('/scan'),
  latestReport: (targetType: string, targetName: string) =>
    http.get<ScanReport>('/scan-reports/latest', {
      params: { targetType, targetName },
      validateStatus: s => s === 200 || s === 204,
    }),
}
```

`validateStatus` es necesario porque axios por defecto trata `204` como éxito igual, pero conviene dejarlo explícito dado que el 204 es parte del contrato (§5.6), no un accidente.

6.2 Hook de polling unificado

Nuevo archivo `frontend/src/hooks/useScanPolling.ts` — un solo hook para los 3 casos, porque los 3 escriben en la misma tabla con la misma convención (§3):

```
// frontend/src/hooks/useScanPolling.ts
import { useCallback, useRef, useState } from 'react'
import { scanApi } from '../services/api'
import type { ScanTargetType } from '../types'

export type ScanPollState = 'idle' | 'running' | 'done' | 'timeout' | 'error'

// Global/Entorno procesan potencialmente muchos mas activos que un scan
// puntual -- se les da mas tiempo antes de declarar timeout.
const TIMEOUT_MS: Record<ScanTargetType, number> = {
  ACTIVO: 30_000,
  ENTORNO: 60_000,
  GLOBAL: 90_000,
}
const POLL_INTERVAL_MS = 3_000

export function useScanPolling() {
  const [state, setState] = useState<ScanPollState>('idle')
  const timerRef = useRef<number | null>(null)
```

```

const stop = useCallback(() => {
  if (timerRef.current) window.clearTimeout(timerRef.current)
  timerRef.current = null
}, [])

const start = useCallback((
  targetType: ScanTargetType,
  targetName: string,
  onDone?: () => void,
) => {
  stop()
  setState('running')
  const baselineAt = Date.now()
  const deadline = baselineAt + TIMEOUT_MS[targetType]

  const poll = async () => {
    try {
      const { status, data } = await scanApi.latestReport(targetType, targetName)
      if (status === 200 && data && new Date(data.executedAt).getTime() >= baselineAt) {
        setState('done')
        onDone?.()
        return
      }
    } catch {
      // error de red puntual: se sigue reintentando hasta el timeout,
      // no se corta el polling por un solo fallo transitorio
    }
    if (Date.now() >= deadline) {
      setState('timeout')
      return
    }
  }
  timerRef.current = window.setTimeout(poll, POLL_INTERVAL_MS)
}
timerRef.current = window.setTimeout(poll, POLL_INTERVAL_MS)
}, [stop])

return { state, start, stop }
}

```

Por qué comparar **executedAt** contra un **baseline** y no solo "¿hay algún reporte?". Si ya corrió un scan de ese mismo alcance antes (muy probable — el scheduler diario ya corre un **GLOBAL** todos los días a las 9:00, ver documento de automatización §4.2), **GET /scan-reports/latest** va a devolver ese reporte viejo desde el primer poll. Comparar contra el timestamp de cuando arrancó **este** disparo es lo que distingue "ya terminó el scan que acabo de pedir" de "hay un reporte de ayer".

6.3 Inventario: fix del scan por activo + scan por entorno

`frontend/src/pages/Inventory.tsx` — cambios sobre el archivo actual:

Importar el hook y `scanApi`:

```

import { assetApi, environmentApi, scanApi } from '../services/api'
import { useScanPolling } from '../hooks/useScanPolling'

```

Reemplazar `handleScan` (hoy muestra éxito sin esperar nada):

```

const { state: scanState, start: startPolling } = useScanPolling()

const handleScan = async (asset: Asset) => {
  setScanning(asset.id)
  try {
    await assetApi.triggerScan(asset.id)
  }
}

```

```

    toast('Escaneo disparado, esperando resultado...')
    startPolling('ACTIVO', asset.name, () => {
      toast.success(`Escaneo de "${asset.name}" completo`)
      load()
    })
  } catch (err: unknown) {
    toast.error(err instanceof Error ? err.message : 'Error al disparar el escaneo')
  } finally {
    setScanning(null)
  }
}

```

2. Nota: `handleScan` pasa a recibir el `Asset` completo (no solo el `id`) porque el polling necesita `asset.name` — ajustar el `onClick` de la columna "Acciones" de `() => handleScan(Number(v))` a `() => handleScan(row)`.
3. El estado `scanState` (`'timeout'`) se puede usar para un toast honesto adicional si se quiere distinguir "sigue corriendo" de "no sabemos" — para el alcance de este documento alcanza con no mostrar éxito falso; el detalle de UI de timeout queda a criterio de implementación.
4. Agregar el control de "escanear entorno completo", cerca de la barra de búsqueda:

```

const [envToScan, setEnvToScan] = useState<number | ''>('')
const [scanningEnv, setScanningEnv] = useState(false)

const handleScanEnvironment = async () => {
  if (!envToScan) return
  const env = environments.find(e => e.id === envToScan)
  if (!env) return
  setScanningEnv(true)
  try {
    await scanApi.triggerEnvironment(env.id)
    toast(`Escaneando entorno "${env.name}"...`)
    startPolling('ENTORNO', env.name, () => {
      toast.success(`Escaneo de "${env.name}" completo`)
      load()
    })
  } catch (err: unknown) {
    toast.error(err instanceof Error ? err.message : 'Error al disparar el escaneo del entorno')
  } finally {
    setScanningEnv(false)
  }
}

```

JSX, agregado junto al `<form>` de búsqueda existente:

```

<div className="flex gap-2 mb-5 items-center">
  <select className="input max-w-xs" value={envToScan}
    onChange={e => setEnvToScan(e.target.value ? Number(e.target.value) : '')}>
    <option value="">Escanear entorno...</option>
    {environments.map(env => (
      <option key={env.id} value={env.id}>{env.name}</option>
    ))}
  </select>
  <button onClick={handleScanEnvironment} disabled={!envToScan || scanningEnv} className="btn-ghost">
    {scanningEnv ? 'Escaneando...' : 'Escanear'}
  </button>
</div>

```

6.4 Dashboard: scan global

`frontend/src/pages/Dashboard.tsx` — agregar el botón en `PageHeader` (mismo patrón que "Nuevo Activo" en Inventario):

```
import { scanApi } from '../services/api'
import { useScanPolling } from '../hooks/useScanPolling'
import { useState } from 'react'

// dentro del componente:
const { start: startPolling } = useScanPolling()
const [scanningAll, setScanningAll] = useState(false)

const handleGlobalScan = async () => {
  setScanningAll(true)
  try {
    await scanApi.triggerGlobal()
    toast('Escaneo global disparado...')
    startPolling('GLOBAL', 'TODO', () => {
      toast.success('Escaneo global completo')
      dashboardApi.getStats().then(r => setStats(r.data))
    })
  } catch (err: unknown) {
    toast.error(err instanceof Error ? err.message : 'Error al disparar el escaneo global')
  } finally {
    setScanningAll(false)
  }
}

// JSX:
<PageHeader
  title="Dashboard"
  subtitle="Vista general del estado de seguridad"
  actions={
    <button onClick={handleGlobalScan} disabled={scanningAll} className="btn-primary">
      {scanningAll ? 'Escaneando...' : 'Escanear todo'}
    </button>
  }
/>
```

Por qué "TODO" literal acá y no una variable. Es el único valor de `target_name` que `Persist_Scan_Statistics` escribe para el caso `GLOBAL` (§3) — no es un placeholder de UI, es el contrato real con la base de datos.

7. Procedimiento de implementación

Orden sugerido, cada paso deja el sistema en un estado funcional (se puede probar/commitear incrementalmente):

1. **Backend — contrato base:** §5.1 (`N8nWebhookService`) + §5.2 (excepción + handler). No rompe nada existente todavía porque el método viejo se reemplaza junto con su único caller en el paso siguiente.
2. **Backend — fix scan por activo:** §5.3. Con esto solo, ya se puede probar el caso `ACTIVO` de punta a punta (con la corrección de §5.6 aplicada también, para poder verificar por polling manual).
3. **Backend — lectura:** §5.6 (repository + DTO). Necesario antes de §5.4/5.5 para poder validarlos.
4. **Backend — entorno y global:** §5.4 y §5.5.
5. **Frontend — hook + tipos + API client:** §6.1 y §6.2. No cambia UI todavía.
6. **Frontend — Inventario:** §6.3 (fix del scan por activo + control de entorno).
7. **Frontend — Dashboard:** §6.4 (scan global).

8. Validación (checklist)

■	Verificación	Cómo comprobarlo
■	Scan por activo llega a n8n con el campo correcto	Disparar desde Inventario, revisar la ejecución en <code>http://localhost:5678</code> → el nodo <code>Edit Fields</code> debe mostrar <code>filtro_activo</code> con el nombre del activo, no <code>undefined</code>
■	Scan por activo detecta vulnerabilidades reales	<code>SELECT * FROM vulnerabilities_audit WHERE asset_id = <id> ORDER BY detected_at DESC</code> tras el scan
■	Scan por entorno dispara para todos los activos del entorno	Revisar <code>system_errors</code> y <code>vulnerabilities_audit</code> de más de un activo del mismo entorno tras un solo disparo
■	Scan global no filtra nada	<code>SELECT target_type, target_name FROM scan_reports ORDER BY executed_at DESC LIMIT 1</code> → <code>GLOBAL / TODOS</code>
■	El polling detecta el fin del scan	Disparar un scan y confirmar que el toast de "completo" aparece solo, sin refrescar la página, dentro del timeout correspondiente
■	El polling no confunde un scan viejo con el nuevo	Disparar dos scans del mismo alcance seguidos; el segundo toast de "completo" debe aparecer después de que termine el segundo, no inmediatamente por el reporte del primero
■	Error de n8n se ve como error, no como éxito	Bajar el contenedor de n8n (<code>docker compose stop n8n</code>) y disparar un scan → el frontend debe mostrar un toast de error (502), no de éxito
■	<code>GET /scan-reports/latest</code> sin reportes previos no rompe nada	Probar contra un <code>target_name</code> que nunca corrió → <code>204</code> , el frontend sigue esperando en vez de tirar error

9. Resolución de problemas

Síntoma	Causa probable	Solución
El scan por activo sigue sin detectar nada tras el fix	El nombre del activo (<code>assets.name</code>) no coincide exactamente con como lo interpreta el nodo <code>Check_Internal_Assets</code> (sensible a mayúsculas/espacios)	Verificar el valor exacto de <code>a.name</code> en la base contra lo que llega en <code>filtro_activo</code> en la ejecución de n8n
El polling nunca termina (siempre <code>timeout</code>)	El scheduler diario ya corrió un <code>GLOBAL</code> hoy y el reloj del navegador/servidor está desfasado, haciendo que <code>executedAt >= baselineAt</code> nunca sea cierto por un problema de zona horaria	Confirmar que backend y frontend interpretan las fechas en el mismo huso horario (ideal: comparar todo en UTC/epoch, no depender de <code>GENERIC_TIMEZONE</code> para esto)
<code>POST /api/environments/{id}/scan</code> devuelve 404	El <code>id</code> de entorno no existe — no es un bug del endpoint nuevo, es <code>ResourceNotFoundException</code> funcionando como se espera	Confirmar el id real con <code>GET /api/environments</code>
Scan global tarda mucho más de 90s y el frontend igual muestra timeout	Sin paginación en la GitHub Advisory API (bug ya documentado en el análisis estratégico como <code>PBN8N03</code>), un scan <code>GLOBAL</code> puede tardar bastante más si en algún momento se corrige esa paginación	Ajustar <code>TIMEOUT_MS.GLOBAL</code> en <code>useScanPolling.ts</code> según el tiempo real observado, no es un valor fijo para siempre
Cualquier scan devuelve 502 con <code>"webhook ... is not registered"</code> segundos después de <code>docker compose up</code> en un volumen nuevo	Verificado contra Docker real: el router de webhooks de n8n no queda montado en el mismo instante en que el proceso final arranca (<code>exec n8n start</code> al final de <code>bootstrap.sh</code>) — condición de carrera de arranque, misma familia que la ya documentada para <code>/rest/settings</code> en el documento de automatización de n8n (§4.2 de ese documento). Reintentando ~8s más tarde el mismo scan respondió 200 sin ningún otro cambio	No es un bug de esta implementación (el 502 es exactamente el comportamiento correcto ante un webhook no disponible). Si se repite seguido en la demo, esperar unos segundos tras ver "Bootstrap inicial completo." en los logs antes del primer scan; una solución de fondo sería que el healthcheck de n8n en <code>docker-compose.yml</code> también sondee el endpoint del webhook, no solo <code>/rest/settings</code>

10. Fuera de alcance / mejoras futuras

- **MTTR y el bug de `countByPriority("CRITICAL")`** (§2) — no se tocan acá; son del dominio de negocio del Dashboard, no del disparo/seguimiento de scans.
- **No revertir `lastScan` si `n8n` falla** (nota de §5.2) — requeriría mover el trigger del webhook *antes* de confirmar la actualización del activo, con manejo transaccional más cuidadoso. Vale la pena si en la demo se observa que el estado queda inconsistente, no antes.
- **Paginación de GitHub Advisory API** (`PBN8N03`, ya en el backlog del análisis estratégico) — afecta directamente cuánto puede tardar un scan `GLOBAL` o `ENTORNO`; cuando se corrija, revisar los timeouts de `useScanPolling.ts`.
- **WebSocket/SSE en vez de polling** — sigue sin justificarse para este volumen de eventos, mismo argumento que ya daba la sección 7 del documento de automatización de `n8n`.
- **Página dedicada de Entornos** — hoy no existe; el control de "escanear entorno" se agregó a Inventario por ser el lugar donde ya se cargan los entornos. Si en el futuro se crea una página de Entornos propia, este control se puede mover ahí sin cambios en el backend.

11. Conclusión

Los 3 tipos de scan no requieren ningún cambio en n8n ni en la base de datos — el workflow y el esquema ya están preparados (evidencia en la sección 3). Todo el trabajo es de plomería en backend (3 endpoints + 1 de lectura, todos delgados) y frontend (un hook de polling reutilizado 3 veces). El punto de mayor riesgo real no es la lógica nueva sino el fix del payload existente (§5.3): sin él, ninguno de los 3 casos funciona, porque es el mismo `N8nWebhookService` el que los sirve a todos.

Estado al aplicar este documento: los 3 tipos de scan disparan y se pueden seguir desde la UI sin entrar a n8n, usando el mismo mecanismo de lectura para los 3 casos — consistente con el objetivo de desacoplar el frontend de la interfaz de n8n que ya fijaba el documento de automatización.